

RUNBOOK OPERASIONAL — KidzPlayful

Prosedur yang dijalankan **saat kejadian**. Dibuat terpisah dari `INFRASTRUKTUR-KIDZPLAYFUL.md` dengan alasan sederhana: runbook dibaca saat panik, dan tidak boleh terkubur di dokumen 2.000 baris.

Cetak PDF-nya dan simpan offline. Saat situs mati atau database tidak bisa diakses, dokumen yang hanya ada di dalam aplikasi tidak menolong.

Disusun: 31 Juli 2026 · **Status:** RB-01 s/d RB-02 belum pernah dijalankan — lihat catatan di masing-masing.

Daftar prosedur

Kode	Prosedur	Kapan dipakai	Frekuensi
RB-01	Backup mingguan	Tiap Jumat	rutin
RB-02	Uji restore	Sekali sekarang, lalu bulanan	rutin
RB-03	DR: database tidak bisa diakses	Insiden	—
RB-04	DR: data terhapus / rusak	Insiden	—
RB-05	Insiden: situs down	Alert PAGE	—
RB-06	Rilis dengan migrasi	Tiap rilis berskema	rutin
RB-07	Rollback deploy	Rilis rusak	—
RB-08	Rotasi kredensial bocor	Insiden keamanan	—
RB-09	Bersihkan data uji di produksi	Berkala	rutin

Prasyarat sekali-pasang

Sebelum runbook ini bisa dijalankan, tiga hal harus ada. **Ketiganya belum ada** per 31 Juli 2026:

#	Prasyarat	Untuk
1	<code>postgresql-client</code> (versi $\geq$ server) + 7-Zip di PATH	RB-01, RB-02
2	Berkas rahasia di luar repo : <code>~/.kidzplayful-backup.env</code> (chmod 600) berisi <code>PGURI</code> dan <code>PASSPHRASE</code> . Passphrase disimpan di password manager.	RB-01
3	Supabase Pro (backup harian otomatis; Free tidak punya backup sama sekali)	RB-03, RB-04

Larangan tegas: berkas backup tidak boleh masuk repo. Tambahkan ke `.gitignore` : `*.dump` , `*.7z` , `backup/` , `.kidzplayful-backup.env` . Repo ini publik.

RB-01 — Backup mingguan

Kapan: tiap Jumat, ± 10 menit. **Prasyarat:** 1 & 2 di atas.

Dua hal harus di-backup, dan ini poin yang sering terlewat: **backup database Supabase TIDAK mencakup berkas Storage**. Bukti bayar orang tua dan nota keuangan berada di Storage, jadi butuh jalur terpisah.

Langkah

1. Jalankan `bash scripts/backup-db.sh` → menghasilkan satu `.7z` terenkripsi.
2. Jalankan `bash scripts/backup-storage.sh` → sinkron inkremental bucket.
3. Unggah `.7z` ke Google Drive folder `kidzplayful-backup` .
4. Sebulan sekali: salin juga ke disk eksternal (offline — aman dari ransomware dan dari akun yang hilang).

scripts/backup-db.sh

```
#!/usr/bin/env bash
# scripts/backup-db.sh – backup mandiri. Jalankan dari laptop owner tiap Jumat.
#
# CATATAN KONEKSI: pakai port 5432 (session pooler / direct). Port 6543 (transaction
# pooler) TIDAK mendukung pg_dump. Koneksi direct kini IPv6-only – bila jaringan
# IPv4-only, pakai hostname pooler, atau add-on IPv4 (~$4/bln 🌐).
set -euo pipefail
source ~/.kidzplayful-backup.env      # PGURI, PASSPHRASE

STAMP=$(date +%Y%m%d-%H%M)
DIR="/d/backup-kidzplayful/$STAMP"
mkdir -p "$DIR"

echo "[1/5] schema public (struktur + data)"
pg_dump "$PGURI" --format=custom --no-owner --no-privileges --schema=public --
file="$DIR/public.dump"

echo "[2/5] schema auth (akun ortu – TANPA INI, restore = semua user hilang)"
pg_dump "$PGURI" --format=custom --no-owner --no-privileges --schema=auth --file="$DIR/auth.dump"

echo "[3/5] schema storage (metadata objek; berkas fisik lihat backup-storage.sh)"
pg_dump "$PGURI" --format=custom --no-owner --no-privileges --schema=storage --
file="$DIR/storage-meta.dump"

echo "[4/5] manifest verifikasi (dipakai saat uji restore RB-02)"
psql "$PGURI" -Atc "
    select 'profiles='||(select count(*) from profiles)
        || ' anak='||(select count(*) from anak)
        || ' hasil_main='||(select count(*) from hasil_main)
        || ' transaksi_keuangan='||(select count(*) from transaksi_keuangan)
        || ' pendaftaran_event='||(select count(*) from pendaftaran_event)
        || ' pesanan='||(select count(*) from pesanan)
        || ' total_masuk='||(select coalesce(sum(jumlah),0) from transaksi_keuangan where
arah='masuk')
        || ' total_keluar='||(select coalesce(sum(jumlah),0) from transaksi_keuangan where
arah='keluar')
        || ' migrasi='||(select coalesce(max(versi),'-') from schema_migrations)
" > "$DIR/MANIFEST.txt"
cat "$DIR/MANIFEST.txt"

echo "[5/5] enkripsi (isinya data pribadi anak & bukti transfer)"
7z a -t7z -mhe=on -p"$PASSPHRASE" "$DIR.7z" "$DIR" >/dev/null
rm -rf "$DIR"
echo "SELESAI: $DIR.7z ($(du -h "$DIR.7z" | cut -f1))"
```

scripts/backup-storage.sh

```
#!/usr/bin/env bash
# scripts/backup-storage.sh – bucket Storage TIDAK tercakup backup DB Supabase.
# Supabase Storage mendukung protokol S3 → rclone (inkremental, murah).
#   rclone config: tipe s3, provider Other,
#   endpoint = https://<ref>.supabase.co/storage/v1/s3 • region = ap-south-1
#   access_key/secret = S3 Access Keys dari Dashboard → Storage → S3
set -euo pipefail
for B in aset privat; do
  echo "== bucket $B =="
  rclone sync "supabase:$B" "/d/backup-kidzplayful/$B" --progress --transfers 8 --checksum
done
du -sh /d/backup-kidzplayful/aset /d/backup-kidzplayful/privat
```

Verifikasi berhasil

- Berkas `.7z` ada, ukurannya wajar (bukan beberapa KB), dan `MANIFEST.txt` tercetak dengan angka yang masuk akal.
- Angka di manifest **tidak menurun** dibanding minggu lalu (kecuali ada penghapusan yang disengaja — kalau menurun tanpa alasan, itu sendiri sebuah temuan).

Retensi

Mingguan 8 minggu · bulanan 12 bulan · tahunan permanen (data keuangan untuk kebutuhan pembukuan).

RB-02 — Uji restore

Backup tanpa uji restore bukan backup. Ini prosedur terpenting di seluruh runbook, dan **belum pernah dijalankan sekali pun**. Uji pertama masuk P0.

Jadwal: sekali sekarang (ke Postgres lokal via Docker, ~2 jam) → bulanan Sabtu pertama (ke proyek Supabase kedua, ~1,5 jam) → kuartalan (restore + jalankan aplikasi + smoke 5 alur, ~3 jam).

Checklist wajib — centang semua, dan catat waktunya

1. `pg_restore` selesai tanpa error fatal.
2. Jalankan query yang sama seperti `MANIFEST.txt` pada database hasil restore → **8 angka harus identik**, termasuk `total_masuk` dan `total_keluar`.
3. `select max(versi) from schema_migrations` sama dengan manifest.
4. Login satu akun orang tua uji berhasil (membuktikan `auth.users` **dan** `profiles` ikut ter-restore — inilah gunanya mem-backup schema `auth`).
5. Buka `/pilih-anak`, `/main/<anakId>`, `/admin/keuangan` → tidak ada halaman error.
6. Buka satu URL `bukti/` dari backup Storage → berkasnya ada.

7. **Catat total menit dari mulai sampai langkah 6 selesai. Angka itu adalah RTO nyata Anda** — bukan angka yang tertulis di tabel mana pun. Tulis di tabel "Riwayat uji restore" di [dokumen infrastruktur §10](#).

Kalau gagal

Gagal pada uji restore adalah **kabar baik** — ditemukan saat tidak ada tekanan. Catat penyebabnya, perbaiki skrip backup, lalu ulangi dari langkah 1. Jangan tandai RB-01 sebagai "beres" sebelum RB-02 pernah lulus.

RB-03 — DR: database tidak bisa diakses

Gejala: `/api/health/db` balas 503, atau seluruh halaman error, atau dashboard Supabase menunjukkan proyek jeda/CPU 100%.

Langkah

1. **(2 menit) Pastikan ini bukan masalah aplikasi.** Cek `/api/health` (tanpa DB) — kalau ia 200 sementara `/api/health/db` 503, masalahnya di database, bukan di Vercel.
2. **(3 menit) Cek penyebab yang paling sering, berurutan:** - Supabase → Reports → Database: CPU, Disk IO budget, jumlah koneksi. **Disk IO budget habis** membuat seluruh database melambat tanpa error yang jelas. - Supabase → Settings → Usage: apakah ada batas terlampaui (ukuran DB, egress) yang memicu pembatasan. - Free tier: apakah proyek **dijeda karena idle** (terjadi setelah ~7 hari tanpa aktivitas). - Halaman status Supabase: apakah ini gangguan penyedia.
3. **(5 menit) Bila CPU/IO jenuh karena query:** Supabase → Reports → Query Performance, cari query terlama. Tersangka utama berdasarkan audit: halaman `/admin/keuangan/*` yang melakukan full scan `transaksi_keuangan`, dan `catatHasilCore` yang menarik seluruh riwayat `hasil_main`. **Mitigasi cepat:** naikan tingkat compute satu tingkat (efektif dalam beberapa menit) — itu membeli waktu, bukan memperbaiki akar masalah.
4. **Bila database rusak, bukan hanya lambat:** lanjut ke [RB-04](#).
5. **(10 menit) Setelah pulih:** pantau `/api/health/db` dan Sentry selama 30 menit sebelum menyatakan selesai.

RB-04 — DR: data terhapus atau rusak

Penyebab paling mungkin di platform ini: perintah salah di SQL Editor produksi. Ini bukan spekulasi — 86 migrasi dijalankan manual di sana, jadi frekuensi tangan manusia di lingkungan produksi tinggi.

0. (2 menit) Bekukan kerusakan — lakukan ini SEBELUM berpikir

- Aktifkan Vercel Deployment Protection, **atau** set env `NEXT_PUBLIC_MODE_PEMELIHARAAN=1` lalu redeploy. Tujuannya menghentikan tulis baru yang akan menimpa keadaan yang masih bisa dipulihkan.

- **Catat waktu insiden** (WIB dan UTC). Angka ini menentukan titik PITR.
- Jangan menjalankan perintah perbaikan spekulatif di produksi. Setiap tulis tambahan mempersempit pilihan.

1. (5 menit) Tentukan ruang lingkup

Ruang lingkup	Jalur
Satu tabel / beberapa baris	Jalur A — restore selektif
Seluruh database	Jalur B — restore penuh
Berkas Storage hilang	Jalur C

Jalur A — selektif (30–60 menit)

1. Restore dump terakhir ke Postgres lokal (Docker) — **jangan** ke produksi.
2. Ekspor hanya yang terdampak: `pg_dump --table=<tabel> --data-only` .
3. Impor ke produksi **di dalam transaksi**; verifikasi `count(*)` sebelum `COMMIT` .
4. Untuk `transaksi_keuangan` : bandingkan `sum(jumlah)` per arah sebelum dan sesudah, dan cocokkan dengan `MANIFEST.txt` .

Jalur B — penuh (2–6 jam)

Kondisi	Cara
Pro + PITR	Dashboard → Database → Restore to point in time → 5 menit sebelum waktu insiden
Pro tanpa PITR	Dashboard → Database → Backups → pilih tanggal → Restore
Free (tanpa keduanya)	Buat proyek baru → <code>pg_restore</code> <code>public</code> + <code>auth</code> + <code>storage-meta</code> → perbarui env Vercel (URL, anon key, service role) → redeploy → perbarui Auth URL Configuration

Lalu jalankan checklist verifikasi [RB-02](#) langkah 2–6.

Jalur C — Storage (30 menit)

```

rclone copy /d/backup-kidzplayful/aset supabase:aset --immutable
rclone copy /d/backup-kidzplayful/privat supabase:privat --immutable

```

`--immutable` mencegah menimpa berkas yang sudah ada — jadi hanya yang benar-benar hilang yang dipulihkan.

3. (10 menit) Buka kembali

Nonaktifkan mode pemeliharaan. Pantau `/api/health/db` + Sentry 30 menit.

4. (esok hari) Post-mortem

Tulis di [dokumen infrastruktur §10](#): penyebab, **RPO nyata**, **RTO nyata**, dan **satu** perubahan pencegah. Satu saja — yang benar-benar akan dikerjakan.

RB-05 — Insiden: situs down

Pemicu: alert PAGE "Situs mati" (`/api/health` gagal 2× berturut-turut).

1. **(1 menit)** Buka situs dari jaringan lain (data seluler). Kalau normal, masalahnya di jaringan Anda atau DNS lokal — bukan insiden.
2. **(2 menit)** Vercel → Deployments: apakah ada deploy baru tepat sebelum alert? Kalau ya → [RB-07](#). Ini penyebab paling sering dan paling cepat diperbaiki.
3. **(2 menit)** Vercel → Observability → Runtime Logs: cari error berulang. Lalu Sentry: apakah ada issue baru dengan lonjakan tajam?
4. **(2 menit)** Cek `/api/health` vs `/api/health/db` untuk memisahkan masalah aplikasi dari masalah database → bila database, lanjut [RB-03](#).
5. **(3 menit)** Cek halaman status Vercel dan Supabase. Bila gangguan penyedia: tidak ada yang bisa dikerjakan selain komunikasi — pasang pesan di kanal WhatsApp orang tua bila lebih dari 30 menit.
6. **(5 menit)** Kalau penyebabnya kuota/tagihan (Spend Management memblokir), naikkan batas. Inilah alasan auto-pause **tidak** diaktifkan untuk produksi.
7. **Setelah pulih:** catat di riwayat insiden.

RB-06 — Rilis dengan migrasi

Prosedur ini menutup **penyebab insiden yang sudah pernah terjadi**: Vercel auto-deploy tiap push `master`, sementara migrasi dijalankan manual sesudahnya — sehingga kode selalu mendahului skema. Insiden `kuota_*` adalah wujudnya.

Pola wajib: expand → migrate → contract

Fase	Yang dilakukan	Aturan
1. EXPAND	Migrasi saja : tambah kolom nullable/berdefault, tabel baru, index baru	Tanpa perubahan kode di rilis ini. Selalu kompatibel ke belakang.
2. DEPLOY	Kode yang menulis & membaca kolom baru	Dipush setelah fase 1 terbukti. Kode toleran terhadap <code>null</code> , bukan terhadap "kolom tidak ada".
3. BACKFILL	Isi data lama, bertahap (<code>limit</code>), idempoten	Jangan <code>update</code> satu tabel besar dalam satu perintah.
4. CONTRACT	<code>set not null</code> , buang kolom lama, buang lapisan toleransi	Minimal 1 rilis setelah fase 3, dan hanya bila stabil ≥ 1 minggu.

Checklist tiap rilis

1. Migrasi baru mengikuti template idempoten (`if not exists` , `drop policy if exists` sebelum `create policy` , mencatat diri ke `schema_migrations`).
2. CI hijau, termasuk gerbang "**migrasi dijalankan dua kali harus no-op**".
3. Jalankan migrasi di **beta** lebih dulu (bila environment beta sudah ada), lalu produksi.
4. Jalankan `node tools/migrate.mjs --db-url "$PGURI"` — tempel keluarannya di PR.
5. Naikkan env `MIGRASI_MINIMAL` di Vercel ke versi terbaru.
6. Push → tunggu deploy → cek `/api/health/db` = 200 dengan `cek.migrasi.ok = true` .
7. Pantau Sentry 15 menit.

Kapan pola "kolom toleran" tetap dipakai

- **Ya** — selama jendela fase 2 (kolom sudah ada tapi masih `null` untuk baris lama); untuk field opsional/kosmetik; dan untuk kompatibilitas aplikasi mobile yang versinya tertinggal di HP pengguna (**alasan permanen dan sah** — rilis app store tidak bisa dipaksa).
- **Tidak** — sebagai satu-satunya pengaman race; di jalur keuangan, di mana kegagalan **harus** berbunyi; dan sebagai alasan `catch {}` kosong.

RB-07 — Rollback deploy

Kapan: rilis baru menyebabkan error massal atau situs down.

1. **(1 menit)** Vercel → Deployments → pilih deployment terakhir yang sehat → **Promote to Production**. Ini instan; jangan mencoba memperbaiki maju-maju saat pengguna sedang terdampak.
2. **(1 menit)** Verifikasi situs normal, lalu beri tahu bila ada yang sudah mengeluh.
3. ⚠️ **Periksa apakah rilis itu menyertakan migrasi.** Ini bagian yang paling mudah salah: - **Migrasi hanya menambah** (kolom nullable, tabel, index) → **jangan di-rollback**. Skema yang lebih maju aman untuk kode lama, dan mengembalikannya justru berisiko. - **Migrasi menghapus/mengetatkan** (`drop column` , `set not null`) → kode lama bisa gagal. Di sinilah pola expand→contract [RB-06](#) membayar dirinya: bila diikuti, situasi ini tidak pernah terjadi.
4. **(catatan)** "Server Action was not found on the server" **setelah** deploy bukan bug — itu klien lama memegang ID Server Action lama. Cukup muat ulang keras (Ctrl+Shift+R). Jangan me-rollback karena ini.
5. Perbaiki akar masalah di branch, lewat PR.

RB-08 — Rotasi kredensial yang bocor

Status per 31 Juli 2026: prosedur ini perlu dijalankan sekarang. Ada 56 kemunculan email + kata sandi admin produksi di 23 skrip `tools/*.mjs` pada repo publik. Asumsikan sudah bocor —

pencarian kode GitHub mengindeks repo publik dalam hitungan menit.

Langkah, berurutan

#	Langkah	Detail
1	Rotasi kata sandi (15 menit)	Supabase → Auth → Users → akun admin → set kata sandi baru (generator, ≥ 24 karakter) → simpan hanya di password manager
2	Audit jejak penyalahgunaan (1 jam)	<code>auth.users.last_sign_in_at</code> akun admin (ada login yang tidak Anda kenali?) · <code>select * from aktivitas order by dibuat_at desc limit 200</code> · baris <code>transaksi_keuangan</code> / <code>pesanan</code> / <code>postingan</code> yang asing · <code>select name, created_at from storage.objects order by created_at desc limit 100</code> (berkas asing) · dan <code>git log -p -S 'service_role'</code> — anon key aman dipublikasikan, service role tidak
3	Ganti identitas admin (30 menit)	Buat akun admin baru dengan email yang tidak mudah diduga (bukan pola <code>admin@<domain></code>). Turunkan akun lama menjadi non-admin — jangan dihapus , ada foreign key <code>dibuat_oleh</code> yang merujuknya
4	Kredensial → env var (2–3 jam)	Skrip membaca <code>KP_EMAIL</code> , <code>KP_PASSWORD</code> , <code>KP_BASE_URL</code> , <code>KP_SUPABASE_URL</code> ; fail-fast bila kosong; nilai di <code>.env.tools.local</code> (gitignored). Buat <code>tools/_env.mjs</code> bersama agar tidak menyalin 23×
5	Guard anti-produksi (1 jam)	Di <code>tools/_env.mjs</code> : bila URL mengandung domain produksi → <code>process.exit(1)</code> kecuali <code>KP_ALLOW_PROD=1</code> diset eksplisit. Ini yang menutup 11 skrip yang menunjuk localhost tapi memakai Supabase produksi — kategori paling berbahaya karena tampak aman
6	Gitleaks di CI + pre-commit (1 jam)	Gagalkan PR bila ada secret
7	Riwayat git (opsional, nilai rendah)	Kata sandi tetap ada di riwayat walau berkasnya diubah. <code>git filter-repo</code> + force-push mungkin dilakukan (repo publik tanpa kolaborator), tetapi fork dan cache GitHub tetap menyimpannya → rotasi di langkah 1 adalah satu-satunya mitigasi nyata . Jangan menunda langkah 1 demi mengerjakan langkah ini

Aturan permanen sesudahnya

Tidak ada kredensial apa pun di repo. Semua `.env*` di `.gitignore`. Service role key hanya di env Vercel dan di berkas rahasia lokal. Anon key boleh publik **karena dilindungi RLS** — yang berarti RLS harus diperlakukan sebagai lapis pertahanan sesungguhnya, terutama selama repo masih publik dan seluruh skema serta policy bisa dibaca siapa pun.

RB-09 — Bersihkan data uji di produksi

Kapan: setelah menjalankan skrip `tools/*_check.mjs`, dan sebagai pemeriksaan berkala sampai skrip sudah diarahkan ke environment beta.

Masalahnya: 10 skrip menargetkan URL produksi dan benar-benar membuat data nyata di sana — produk uji, pendaftaran uji, dan berkas bukti bayar yang terunggah ke bucket produksi. Pembersihan saat ini hanya berupa konvensi manual, tanpa mekanisme penegak.

Langkah

- 1. **Inventarisasi dulu, jangan langsung hapus.** Cari data bertanda uji: `sql select 'produk' t, id, nama, created_at from public.produk where nama ilike '%uji%' or nama ilike '%test%' union all select 'event', id, judul, created_at from public.event where judul ilike '%uji%' or judul ilike '%test%'; select id, email, created_at from auth.users where email ilike '%uji%' or email ilike '%test%' or email like '%@example.%'; select name, created_at, (metadata->>'size')::bigint from storage.objects where bucket_id = 'aset' and (name like 'uji/%' or name ilike '%test%') order by created_at desc;`
- 2. **Tinjau manual.** Pastikan tidak ada data pelanggan sungguhan yang kebetulan mengandung kata "uji" atau "test".
- 3. **Hapus berurutan** dari anak ke induk (hormati foreign key): `item_pesanan` → `pesanan` , `pendaftaran_event` → `event` , lalu berkas Storage.
- 4. **Periksa efek sampingnya di keuangan.** Pesanan/pendaftaran uji yang pernah diverifikasi kemungkinan sudah membuat baris `transaksi_keuangan` — kalau tidak dibersihkan, laporan keuangan ikut salah. Cocokkan lewat `ref_tipe` / `ref_id` .
- 5. **Konvensi ke depan:** semua data buatan skrip diberi prefiks `[UJI]` dan berkas diletakkan di folder `uji/` , supaya bisa dihapus dengan aman dan otomatis.

Kontak & eskalasi

Situasi	Tindakan
Situs down > 30 menit	Kabari orang tua lewat kanal WhatsApp; pasang pesan di halaman bila memungkinkan
Data pribadi bocor	Catat lingkup & waktu; rotasi kredensial terkait (RB-08); pertimbangkan kewajiban pemberitahuan menurut UU PDP
Gangguan penyedia (Vercel/Supabase)	Pantau halaman status; tidak ada mitigasi teknis; fokus ke komunikasi
Tagihan melonjak	Jangan matikan produksi; naikan batas lalu telusuri penyebabnya dengan <code>ringkas_penyimpanan()</code>